06_Hash Tables

🧠 Concept（中英双语解释）

在编程里，我们经常需要“通过 A 找到 B”：

- 通过“人名”找到“电话号码”
- 通过“单词”找到“定义”
- 通过“账号”找到“密码”

这个 **A** 就是 **Key（键）**，这个 **B** 就是 **Value（值）**。

散列表 (Hash Tables): 一种通过“散列函数”将键（Key）映射到值（Value）的数据结构。
A data structure that maps keys to values using a hash function.

散列函数 (Hash Function): 一个特殊的函数，你给它一个字符串，它返还给你一个数字。它能保证：同样的输入，永远得到同样的数字。 A function where you put in a string and you get back a number. It always maps the same string to the same index.
确定性：每次输入 "apple"，它必须吐出同一个数字（比如 3）。
映射性：不同的输入，应该尽量吐出不同的数字。

在 Python 语言里，这种数据结构的官方名字叫 **“字典” (Dictionary)**。

```
# 创建一个散列表（在Python里叫字典 dict）
book = {}

# 插入键值对
book["apple"] = 0.67  # "apple" 是 Key, 0.67 是 Value
book["milk"] = 1.49   # "milk" 是 Key, 1.49 是 Value

# 查找
print(book["apple"]) # 瞬间得到 0.67
```

🧠 Concept（中英双语解释）

冲突 (Collision)

当两个不同的键（Key）通过散列函数指向同一个位置时，就发生了冲突。
When two different keys are assigned to the same slot by a hash function.

- 解决方法：链表法 (Chaining)
  如果两个不同的 Key（苹果和桃子）都被散列函数分到了 **5 号柜子**，计算机的处理方式是：
  1. 在 5 号格子里不直接放价格。

2. 而是放一个**链表（Linked List）**，像一串糖葫芦一样把数据串起来。

装填因子 (Load Factor)

衡量散列表有多满的比例。 A measure that decides when to increase the hash table capacity.

$$\text{装填因子} = \frac{\text{散列表已包含的元素数}}{\text{总格子数}}$$

- 如果你有 100 个格子，存了 70 个东西，装填因子就是 **0.7**。
- **经验之谈**：一旦装填因子大于 **0.7**，你就该买个更大的“柜子”了（这叫 **调整长度 Resizing**）。把旧柜子里的所有东西**重新算一遍位置**再放进去。

复杂度

理想情况：$O(1)$瞬间找到。是一次计算而非搜索。不随数据量增加而增加。
最坏情况：所有东西都挤在一个柜子里，速度退化为$O(n)$

实际应用

- 1.防止重复
  办投票，每人只能投一次。用散列表，把名字当key，新来一个人，直接查 `voted["张三"]`。由于是 $O(1)$，无论有一万个人还是十亿个人，查重都是**瞬间**完成。
-
  2. 模拟映射
     比如你想把“域名”映射到“IP地址”：
     `Key`：https://www.google.com/search?q=google.com → `Value`：172.217.0.46
     这就是互联网能跑起来的基础（DNS解析）。
-
  3. 缓存 (Caching)
     当你访问一个网页时，服务器会把复杂的计算结果存在散列表里。下次你再访问，它直接从散列表里“秒回”给你，而不需要重新计算。

```
voted = {} # 创建一个空散列表

def check_voter(name):
    if voted.get(name): # 如果在散列表里找到了这个名字
        print("踢出去！他投过了！")
    else:
        voted[name] = True # 把名字存进去，标记为已投票
        print("请进，投出你神圣的一票。")

check_voter("Tom")
```

```
check_voter("Alice")
check_voter("Tom") # 再次调用 Tom
```